Organisation de l'application avec NavigationStack et TabView

Notes de cours — Programmation mobile iOS

1. Pourquoi organiser l'application ?

Une application mobile contient souvent plusieurs grandes sections.

Exemple :

- Accueil
- Produits
- Favoris
- Profil
- Paramètres

On peut utiliser :

```
TabView
```

pour passer entre les grandes sections de l'application, et :

```
NavigationStack
```

pour naviguer à l'intérieur d'une section.

2. Deux types de navigation

Il faut distinguer :

Type de navigation	Outil
`Navigation principale`	TabView
`Navigation hiérarchique`	NavigationStack

Ces deux outils se combinent généralement plutôt que de s'opposer.

3. TabView

Un TabView contient plusieurs vues.

Exemple :

```
struct ContentView: View {
    var body: some View {
        TabView {
            Text("Accueil")
            Text("Produits")
            Text("Profil")
        }
    }
}
```

Mais sans configuration supplémentaire, les onglets ne sont pas très explicites.

4. Ajouter des onglets

On utilise généralement :

```
.tabItem
```

Exemple :

```
TabView {
    Text("Accueil")
        .tabItem {
            Label(
                "Accueil",
                systemImage: "house"
            )
        }
    Text("Profil")
        .tabItem {
            Label(
                "Profil",
                systemImage: "person"
            )
        }
}
```

5. Label

Label permet de combiner une icône et un texte.

Exemple :

```
Label(
    "Accueil",
    systemImage: "house"
)
```

ou :

```
Label(
    "Produits",
    systemImage: "cart"
)
```

6. Exemple avec trois onglets

```
struct ContentView: View {
    var body: some View {
        TabView {
            AccueilView()
                .tabItem {
                    Label(
                        "Accueil",
                        systemImage: "house"
                    )
                }
            ProduitsView()
                .tabItem {
                    Label(
                        "Produits",
                        systemImage: "cart"
                    )
                }
            ProfilView()
                .tabItem {
                    Label(
                        "Profil",
                        systemImage: "person"
                    )
                }
        }
    }
}
```

7. Organisation générale

On peut représenter l'application ainsi :

```
TabView
  │
  ├── Accueil
  │
  ├── Produits
  │
  └── Profil
```

Chaque onglet représente une grande section.

8. Ajouter NavigationStack

Supposons que la section Produits doit permettre :

```
Liste des produits
        ↓
   Détail produit
```

On peut placer un NavigationStack dans cette section.

9. Exemple — NavigationStack dans Produits

```swift
struct ProduitsView: View {
    var body: some View {
        NavigationStack {
            List {
                Text("iPhone")
                Text("MacBook")
                Text("AirPods")
            }
            .navigationTitle("Produits")
        }
    }
}
```

10. Combiner NavigationStack et TabView

Une organisation courante est :

```
TabView
  │
  ├── NavigationStack
  │      └── Accueil
  │
  ├── NavigationStack
  │      └── Produits
  │            └── Détail
  │
  └── NavigationStack
         └── Profil
               └── Paramètres
```

Chaque grande section possède ainsi sa propre navigation.

11. Exemple complet — TabView et NavigationStack combinés

```
struct ContentView: View {
    var body: some View {
        TabView {
            NavigationStack {
                AccueilView()
            }
            .tabItem {
                Label(
                    "Accueil",
                    systemImage: "house"
                )
            }
            NavigationStack {
                ProduitsView()
            }
            .tabItem {
                Label(
                    "Produits",
                    systemImage: "cart"
                )
            }
            NavigationStack {
                ProfilView()
            }
            .tabItem {
                Label(
                    "Profil",
                    systemImage: "person"
                )
            }
        }
    }
}
```

12. Pourquoi un NavigationStack par onglet ?

Cela permet de séparer les différentes hiérarchies.

Exemple :

```
Produits
   ↓
Détail produit
```

et :

```
Profil
   ↓
Paramètres
```

sont deux navigations différentes. Cela rend l'organisation de l'application plus claire.

13. Exemple — Onglet Produits

```swift
struct Produit: Identifiable, Hashable {
    let id = UUID()
    let nom: String
    let prix: Double
}
```

14. Liste des produits

```swift
struct ProduitsView: View {
    let produits = [
        Produit(
            nom: "iPhone",
            prix: 1200
        ),
        Produit(
            nom: "MacBook",
            prix: 2200
        ),
        Produit(
            nom: "AirPods",
            prix: 250
        )
    ]

    var body: some View {
        List(produits) { produit in
            NavigationLink(
                value: produit
            ) {
                Text(produit.nom)
            }
        }
        .navigationTitle("Produits")
        .navigationDestination(
            for: Produit.self
        ) { produit in
            DetailProduitView(
                produit: produit
            )
        }
    }
}
```

15. Vue détail

```
struct DetailProduitView: View {
    let produit: Produit

    var body: some View {
        VStack(spacing: 20) {
            Text(produit.nom)
                .font(.title)
            Text("\(produit.prix) $")
                .font(.title2)
        }
        .navigationTitle("Détails")
    }
}
```

16. Onglet Profil

```
struct ProfilView: View {
    var body: some View {
        VStack(spacing: 20) {
            Image(
                systemName: "person.circle"
            )
                .font(.system(size: 70))
            Text("Alex Martin")
            NavigationLink(
                "Paramètres"
            ) {
                ParametresView()
            }
        }
        .navigationTitle("Profil")
    }
}
```

17. Vue Paramètres

```
struct ParametresView: View {
    var body: some View {
        VStack {
            Text("Paramètres")
        }
        .navigationTitle("Paramètres")
    }
}
```

18. Sélectionner un onglet

On peut mémoriser l'onglet actuellement sélectionné.

```
@State private var ongletSelectionne = 0
```

Puis :

```
TabView(
    selection: $ongletSelectionne
) {
    AccueilView()
        .tabItem {
            Label(
                "Accueil",
                systemImage: "house"
            )
        }
        .tag(0)
    ProduitsView()
        .tabItem {
            Label(
                "Produits",
                systemImage: "cart"
            )
        }
        .tag(1)
}
```

19. À quoi sert tag ?

Chaque onglet reçoit une valeur :

```
.tag(0)
.tag(1)
.tag(2)
```

La variable ongletSelectionne indique l'onglet actuellement affiché.

20. Changer d'onglet depuis le code

Grâce à :

```
@State private var ongletSelectionne = 0
```

on peut écrire :

```
Button("Voir les produits") {
    ongletSelectionne = 1
}
```

L'application passe alors automatiquement à l'onglet Produits.

21. Éviter les nombres magiques

Au lieu de :

```
.tag(0)
.tag(1)
.tag(2)
```

on peut utiliser une énumération.

```
enum Onglet {
    case accueil
    case produits
    case profil
}
```

22. Exemple avec une énumération

```
struct ContentView: View {
    @State private var ongletSelectionne:
        Onglet = .accueil

    var body: some View {
        TabView(
            selection: $ongletSelectionne
        ) {
            AccueilView()
                .tabItem {
                    Label(
                        "Accueil",
                        systemImage: "house"
                    )
                }
                .tag(Onglet.accueil)
            ProduitsView()
                .tabItem {
                    Label(
                        "Produits",
                        systemImage: "cart"
                    )
                }
                .tag(Onglet.produits)
            ProfilView()
                .tabItem {
                    Label(
                        "Profil",
                        systemImage: "person"
                    )
                }
                .tag(Onglet.profil)
        }
    }
}
```

Cette approche est plus lisible.

23. badge

Un onglet peut afficher une information supplémentaire.

Par exemple :

```
Text("Panier")
    .tabItem {
        Label(
            "Panier",
            systemImage: "cart"
        )
    }
    .badge(3)
```

Résultat :

```
Panier  ③
```

Cela peut représenter :

- Articles dans le panier
- Messages non lus
- Notifications

24. Exemple de structure d'une application

Une application de commerce pourrait être organisée ainsi :

```
TabView
  │
  ├── Accueil
  │      └── Promotion
  │
  ├── Produits
  │      ├── Liste
  │      └── Détail
  │
  ├── Panier
  │      └── Confirmation
  │
  └── Profil
         ├── Informations
         └── Paramètres
```

25. Organisation des fichiers

On peut également organiser le projet :

```
Models
   └── Produit.swift

ViewModels
   └── ProduitsViewModel.swift

Views
   ├── Accueil
   │      └── AccueilView.swift
   │
   ├── Produits
   │      ├── ProduitsView.swift
   │      └── DetailProduitView.swift
   │
   └── Profil
          ├── ProfilView.swift
          └── ParametresView.swift
```

26. TabView et MVVM

Le TabView organise principalement la navigation principale.

Le ViewModel ne devrait généralement pas contenir tout le code de navigation visuelle.

Par exemple :

```swift
@Observable
class ProduitsViewModel {
    var produits: [Produit] = []

    func chargerProduits() {
        // Logique
    }
}
```

Puis :

```swift
struct ProduitsView: View {
    @State private var viewModel =
        ProduitsViewModel()

    var body: some View {
        List(viewModel.produits) { produit in
            Text(produit.nom)
        }
    }
}
```

La navigation reste dans la View.

27. Quand utiliser TabView ?

Pour rappel : TabView gère la navigation principale (changer de section), tandis que NavigationStack gère la navigation hiérarchique (avancer dans une section).

Utiliser TabView lorsque l'application possède plusieurs sections principales accessibles régulièrement.

Exemple :

- Accueil
- Recherche
- Favoris
- Profil

28. Quand éviter TabView ?

Il n'est pas nécessaire d'utiliser un TabView pour chaque écran.

À éviter :

```
Accueil
   ↓
Détail produit
```

comme deux onglets séparés. Le détail d'un produit devrait plutôt être ouvert avec :

```
NavigationLink
```

29. Bon principe

Outil	Rôle
`TabView`	Grandes sections
`NavigationStack`	Profondeur dans une section

30. À retenir

Outil	Rôle
`TabView`	Navigation principale
`NavigationStack`	Navigation entre les écrans
`NavigationLink`	Ouvrir un écran
`navigationDestination`	Associer une donnée à une destination
`selection + tag`	Contrôler l'onglet sélectionné
`badge`	Afficher une information sur un onglet

Architecture typique :

```
MainView
   ↓
TabView
   ↓
NavigationStack par section
```

31. Exercice — Application de réservation

Créer une application contenant trois onglets :

- Accueil
- Réservations
- Profil

Utiliser :

```
TabView
```

Chaque onglet doit avoir son propre :

```
NavigationStack
```

1. Accueil

Afficher :

```
Console
Bienvenue
```

et un bouton.

2. Réservations

Créer :

```
struct Reservation: Identifiable, Hashable
```

avec :

```
destination
date
prix
```

Afficher plusieurs réservations dans une liste. Un appui sur une réservation doit ouvrir :

```
DetailReservationView
```

avec :

```
NavigationLink
```

3. Profil

Afficher :

- Nom
- Courriel

et permettre d'ouvrir :

```
ParametresView
```

32. Exercice — Changer d'onglet depuis l'accueil

Ajouter :

```
@State
```

pour connaître l'onglet sélectionné.

Depuis l'écran Accueil, ajouter un bouton :

```
Console
Voir mes réservations
```

qui change automatiquement l'onglet actif vers Réservations.